JPA Relationships: Mapping, Ownership and Cardinality

JPA relationships allow Java entities to reference one another while Hibernate translates those object references into foreign keys and join tables in a relational database.

These notes cover:

- Why relationship mapping is required
- Relationship cardinality
- Unidirectional and bidirectional relationships
- Owning and inverse sides
- `@ManyToOne` and `@OneToMany`
- `@OneToOne`
- `@ManyToMany`
- `@JoinColumn`, `mappedBy` and `@JoinTable`
- Maintaining both sides of a relationship
- Relationship insert, update and dirty-checking behaviour

---

1. The Fundamental Problem

Consider the following Java model:

```
class Student {
    Long id;
    String name;
    Department department;
}
```

In Java, the `department` field stores a reference to a `Department` object:

```
student.getDepartment();
```

A relational database cannot place a Java object inside a column. It represents the same relationship using a foreign key:

```
students
--------------------------------
id | name   | department_id
--------------------------------
1  | Rahul  | 10
2  | Aman   | 10
3  | Priya  | 20
```

```
departments
-----------------
id | name
-----------------
10 | Engineering
20 | HR
```

The Java relationship:

```
student.department
```

must be translated into:

```
students.department_id → departments.id
```

> Relationship mapping is the configuration that tells JPA how an object reference should be represented in the database.

Without ORM, an entity might store only the foreign-key value:

```
@Entity
class Student {
```

```
    @Id
    private Long id;

    private String name;
    private Long departmentId;
}
```

With JPA, the object model can contain an actual entity reference:

```
@Entity
class Student {

    @Id
    private Long id;

    private String name;

    @ManyToOne
    private Department department;
}
```

Hibernate then converts the entity reference into the required SQL and foreign-key value.

2. Entity Relationships

Simple entities contain only values that belong directly to that entity:

```
@Entity
class Student {

    @Id
    private Long id;

    private String name;
```

```
    private Integer age;
}
```

Real applications, however, contain connected entities:

```
class Student {
    Department department;
}
```

```
class Order {
    List<OrderItem> items;
}
```

```
class Employee {
    Employee manager;
}
```

```
class User {
    Profile profile;
}
```

The application therefore contains a graph of connected objects rather than a set of isolated objects. These connections are called **relationships** or **associations**.

---

3. Relationship Cardinality

Cardinality answers one question:

> How many entities on one side can be associated with how many entities on the other side?

JPA supports four basic cardinalities:

Relationship	Meaning	Common example
`@OneToOne`	One entity is associated with one other entity	User and Profile
`@OneToMany`	One entity is associated with many entities	Department and Students
`@ManyToOne`	Many entities are associated with one entity	Students and Department
`@ManyToMany`	Many entities on both sides can be associated	Students and Courses

One-to-many and many-to-one are two views of the same relationship

Suppose the Engineering department contains three students:

```
Engineering
├── Rahul
├── Aman
└── Priya
```

From the department's perspective:

```
One Department → Many Students
```

This is `@OneToMany`.

From a student's perspective:

```
Many Students → One Department
```

This is `@ManyToOne`.

The database does not create two separate relationships. Both directions normally describe the same foreign key:

```
students.department_id
```

4. Unidirectional and Bidirectional Relationships

These terms describe navigation in the Java object model. They do not describe how many directions SQL can query.

Unidirectional relationship

Only one entity contains a reference to the other:

```
class Student {
    Department department;
}
```

```
class Department {
}
```

The application can navigate from `Student` to `Department`:

```
student.getDepartment();
```

It cannot navigate directly from `Department` to its students:

```
department.getStudents(); // No such field
```

The direction is:

```
Student → Department
```

Bidirectional relationship

Both entities contain a reference to the other:

```
class Student {
    Department department;
}
```

```
class Department {
    List<Student> students;
}
```

The application can navigate in both directions:

```
student.getDepartment();
department.getStudents();
```

The direction is:

```
Student ↔ Department
```

A bidirectional mapping still represents one database relationship. In this example, there is still only one foreign key:

```
students.department_id
```

Bidirectional fields do not synchronize themselves

These two Java fields are independent:

```
student.setDepartment(engineering);
```

```
engineering.getStudents().add(student);
```

Calling the first statement does not automatically execute the second. It is therefore possible to produce an inconsistent in-memory object graph:

```
student.department = Engineering
engineering.students = []
```

The application must keep both sides synchronized.

5. Owning Side and Inverse Side

When the same database relationship is mapped on two entities, JPA needs one authoritative side.

Owning side

The owning side is the side JPA uses to determine the database value of the relationship.

For a department–student relationship:

```
@ManyToOne
@JoinColumn(name = "department_id")
private Department department;
```

`Student` is the owning side because changing:

```
student.setDepartment(department);
```

changes:

```
students.department_id
```

Owning side does **not** mean:

- The parent entity
- The more important entity
- The entity that conceptually contains the other entity

It means only that this side controls the database representation of the relationship.

Inverse side

The inverse side provides the opposite navigation path but does not independently control the same relationship:

```java
@OneToMany(mappedBy = "department")
private List<Student> students = new ArrayList<>();
```

Here, `Department.students` is the inverse side.

> For database relationship updates, the owning side is authoritative.

---

6. Many-to-One Mapping

Many students can belong to one department:

```
Student N → 1 Department
```

Database design

```
departments
-------------------
id | name
-------------------
1  | Engineering
2  | HR
```

```
students
--------------------------------
id  | name   | department_id
--------------------------------
101 | Rahul  | 1
102 | Priya  | 1
103 | Aman   | 2
```

`students.department_id` is a foreign key that references `departments.id`.

Multiple student rows can contain the same department ID, which creates the many-to-one relationship.

Mapping with `@ManyToOne`

```java
@Entity
public class Student {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

`@ManyToOne` tells JPA that:

- The field references another entity.
- Many `Student` entities may reference the same `Department`.

`@JoinColumn` specifies the foreign-key column used to store the relationship:

```
Student.department
        ↓
students.department_id
```

By default, the join column references the primary key of the target entity.

If:

```java
engineering.getId() == 5L;
student.setDepartment(engineering);
```

then the database relationship becomes:

```
students.department_id = 5
```

Nullable and mandatory relationships

If a student may exist without a department, the foreign-key value may be `NULL`:

```
id | name    | department_id
-----------------------------
1  | Harshit | NULL
```

The default mapping permits this:

```
@ManyToOne
@JoinColumn(name = "department_id")
private Department department;
```

If every student must belong to a department:

```
@ManyToOne(optional = false)
@JoinColumn(
    name = "department_id",
    nullable = false
)
private Department department;
```

The two settings express the same rule at different layers:

Setting	Layer	Meaning
`optional = false`	JPA object/persistence model	The association is required
`nullable = false`	Generated database schema	The foreign-key column must be `NOT NULL`

Insert behaviour

If the department already exists:

```
Department department =
        entityManager.find(Department.class, 1L);
```

```
Student student = new Student();
student.setName("Rahul");
student.setDepartment(department);

entityManager.persist(student);
```

Hibernate logically executes:

```
INSERT INTO students (name, department_id)
VALUES ('Rahul', 1);
```

The existing department is not inserted again. Hibernate stores its identifier as the foreign key.

If both entities are new:

```
Department department = new Department();
department.setName("Engineering");

Student student = new Student();
student.setName("Rahul");
student.setDepartment(department);
```

Without cascading, persist them explicitly in the correct lifecycle:

```
entityManager.persist(department);
entityManager.persist(student);
```

With cascade persist:

```
@ManyToOne(cascade = CascadeType.PERSIST)
private Department department;
```

persisting the student can also persist the new department.

> Setting an entity reference and managing the lifecycle of the referenced entity are separate concerns.

Update behaviour and dirty checking

Suppose Rahul currently belongs to Engineering:

```
student_id = 1
department_id = 10
```

Changing the department of a managed student:

```
Department hr =
        entityManager.find(Department.class, 20L);

student.setDepartment(hr);
```

changes the relationship from department `10` to `20`. During flush, Hibernate can issue:

```
UPDATE students
SET department_id = 20
WHERE id = 1;
```

This is relationship dirty checking.

---

7. One-to-Many Mapping

`@OneToMany` represents the same department–student relationship from the department side:

```
Department 1 → N Students
```

Database design

No additional foreign key is required:

```
students
---------------------------------
id | name   | department_id
---------------------------------
1  | Rahul  | 10
2  | Aman   | 10
3  | Priya  | 10
```

All three students belong to department `10`. The collection in `Department` is an object-side view of these rows.

Mapping with `@OneToMany`

```java
@Entity
class Department {

    @Id
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department")
    private List<Student> students =
            new ArrayList<>();
}
```

`mappedBy = "department"` refers to the name of the Java field in `Student`:

```java
class Student {
    private Department department;
}
```

It does **not** refer to the database column name `department_id`.

The meaning is:

> `Department.students` is the inverse view of the relationship owned by `Student.department`.

Complete bidirectional mapping

```
@Entity
class Student {

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

```
@Entity
class Department {

    @OneToMany(mappedBy = "department")
    private List<Student> students =
            new ArrayList<>();
}
```

This creates two Java navigation paths:

```
student.getDepartment();
department.getStudents();
```

Both paths describe the same foreign key:

```
students.department_id
```

Maintaining both sides

The following statement updates only the `Student` object:

```
student.setDepartment(engineering);
```

The department collection must also be updated:

```
engineering.getStudents().add(student);
```

For removal:

```
engineering.getStudents().remove(student);
student.setDepartment(null);
```

Relationship helper methods

Instead of repeating synchronization logic throughout the application, place it inside the entity:

```
@Entity
public class Department {

    @OneToMany(mappedBy = "department")
    private List<Student> students =
            new ArrayList<>();

    public void addStudent(Student student) {
        students.add(student);
        student.setDepartment(this);
    }

    public void removeStudent(Student student) {
        students.remove(student);
        student.setDepartment(null);
    }
}
```

The application can now use:

```
engineering.addStudent(rahul);
engineering.removeStudent(rahul);
```

This keeps the object graph consistent and ensures the owning side is updated.

Collection dirty checking

Hibernate can detect changes made to a managed relationship collection:

```
Department department =
        entityManager.find(Department.class, 10L);

department.getStudents().remove(student);
```

However, detecting a collection change is not the same as deciding which side controls the foreign key.

Because `Department.students` uses `mappedBy`, it is the inverse side. The owning side must also be changed:

```
student.setDepartment(null);
```

This is why helper methods are important.

---

8. One-to-One Mapping

A one-to-one relationship means that each entity can be associated with at most one entity on the other side:

```
User 1 ↔ 1 Profile
```

Database design

```
users
----------------
id | username
----------------
1  | rahul
```

```
profiles
-------------------------------
id | bio              | user_id
-------------------------------
10 | Software dev     | 1
```

A foreign key alone does not guarantee one-to-one. Without a uniqueness constraint, the database could allow:

```
id | user_id
-------------
10 | 1
11 | 1
12 | 1
```

That would mean one user has many profiles.

The usual relational representation of one-to-one is:

```
Foreign key + unique constraint
```

Mapping with `@OneToOne`

```java
@Entity
class Profile {

    @Id
    @GeneratedValue
    private Long id;

    private String bio;

    @OneToOne
    @JoinColumn(
        name = "user_id",
        unique = true
```

```
    )
    private User user;
}
```

The essential database difference is:

Mapping	Foreign key	Uniqueness
Many-to-one	Present	Repeated values allowed
One-to-one	Present	Foreign-key value must be unique

Owning and inverse sides

`Profile` is the owning side because it contains the join column:

```
@OneToOne
@JoinColumn(name = "user_id")
private User user;
```

The inverse side in `User` is:

```
@Entity
class User {

    @OneToOne(mappedBy = "user")
    private Profile profile;
}
```

Here, `"user"` refers to `Profile.user`.

```
Profile.user  → Owning side
User.profile  → Inverse side
```

Mandatory one-to-one relationship

If a profile must always belong to a user:

```java
@OneToOne(optional = false)
@JoinColumn(
    name = "user_id",
    nullable = false,
    unique = true
)
private User user;
```

Shared primary-key mapping

Some one-to-one entities cannot exist independently. A profile, for example, may use the same identifier as its user:

```
users
----------------
id | username
----------------
10 | rahul
```

```
profiles
----------------
id | bio
----------------
10 | ...
```

In `profiles`, the `id` column acts as both:

- The primary key of `Profile`
- A foreign key referencing `users.id`

JPA can model this with `@MapsId`:

```java
@Entity
class Profile {

    @Id
    private Long id;
```

```
    private String bio;

    @OneToOne
    @MapsId
    @JoinColumn(name = "id")
    private User user;
}
```

`@MapsId` means that the associated user's ID is also used as the profile's ID.

This mapping is useful when the dependent entity has no meaningful identity or lifecycle without its parent.

---

9. Many-to-Many Mapping

A many-to-many relationship allows multiple entities on each side to be connected:

```
Student N ↔ N Course
```

A student can enroll in many courses, and a course can contain many students.

Database design

```
students
------------
1 | Rahul
2 | Priya
```

```
courses
--------------
101 | Java
102 | Spring
103 | SQL
```

Neither table can represent the relationship using one foreign-key column. A third table is required:

```
student_course
-----------------------
student_id | course_id
-----------------------
1          | 101
1          | 102
2          | 101
2          | 103
```

Each row represents one association between one student and one course.

This middle table is called a:

- Join table
- Association table
- Junction table

It contains two foreign keys:

```
student_id → students.id
course_id  → courses.id
```

The pair should normally be unique:

```
(student_id, course_id)
```

This prevents the same enrollment from being inserted more than once.

Owning side with `@JoinTable`

```java
@Entity
public class Student {

    @Id
    private Long id;
```

```
    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns =
            @JoinColumn(name = "student_id"),
        inverseJoinColumns =
            @JoinColumn(name = "course_id"),
        uniqueConstraints =
            @UniqueConstraint(
                columnNames = {
                    "student_id",
                    "course_id"
                }
            )
    )
    private Set<Course> courses =
            new HashSet<>();
}
```

`@JoinTable` describes the middle table:

Attribute	Meaning
`name`	Name of the join table
`joinColumns`	Foreign key pointing to the owning entity
`inverseJoinColumns`	Foreign key pointing to the other entity

`Student.courses` is the owning side. Changes to this collection control rows in `student_course`:

```
student.getCourses().add(javaCourse);
```

Hibernate can then insert:

```
INSERT INTO student_course (student_id, course_id)
```

```
VALUES (1, 101);
```

Inverse side

```
@Entity
public class Course {

    @Id
    private Long id;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students =
            new HashSet<>();
}
```

`mappedBy = "courses"` points to the `courses` field in `Student`.

```
Student.courses → Owning side
Course.students → Inverse side
```

The mapping creates two Java navigation paths:

```
student.getCourses();
course.getStudents();
```

Both represent rows in the same `student_course` table.

Maintaining both sides

Adding a course to only one collection leaves the in-memory model inconsistent:

```
student.getCourses().add(course);
```

Use helper methods:

```java
public void addCourse(Course course) {
    courses.add(course);
    course.getStudents().add(this);
}

public void removeCourse(Course course) {
    courses.remove(course);
    course.getStudents().remove(this);
}
```

Now:

```java
student.addCourse(course);
student.removeCourse(course);
```

keeps both sides synchronized.

---

10. Lombok with JPA Entities

Common Lombok annotations include:

Annotation	Generated code
`@Getter`	Getter methods
`@Setter`	Setter methods
`@NoArgsConstructor`	Constructor without parameters
`@AllArgsConstructor`	Constructor containing all fields
`@RequiredArgsConstructor`	Constructor for `final` and `@NonNull` fields
`@Builder`	Builder-pattern API

Be careful with `@Data` on JPA entities. In addition to getters and setters, it generates `toString()`, `equals()` and `hashCode()`.

When relationships are bidirectional, generated methods may:

- Recursively move from one entity to the other

- Cause a `StackOverflowError`
- Trigger lazy relationship loading unexpectedly
- Compare large relationship collections
- Produce unstable equality or hash-code behaviour for entities

For JPA entities, explicit Lombok annotations are usually safer:

```
@Getter
@Setter
@NoArgsConstructor
@Entity
public class Student {
    // fields
}
```

If `toString()`, `equals()` or `hashCode()` is generated, relationship fields should generally be excluded and entity identity should be designed deliberately.

---

11. Quick Reference

Relationship annotations

Annotation	Typical database representation
`@ManyToOne`	Foreign key on the many side
`@OneToMany(mappedBy = "...")`	Inverse collection over an existing foreign key
`@OneToOne`	Foreign key with a uniqueness constraint
`@ManyToMany`	Join table containing two foreign keys

Ownership rules

Mapping	Typical owning side
Bidirectional one-to-many/many-to-one	The `@ManyToOne` side containing `@JoinColumn`
Bidirectional one-to-one	The side containing `@JoinColumn`

Mapping	Typical owning side
Bidirectional many-to-many	The side containing `@JoinTable`

Core principles

1. Java stores entity references; relational databases store foreign keys.
2. Cardinality describes how many entities can be associated on each side.
3. Unidirectional and bidirectional describe Java navigation.
4. A bidirectional mapping still represents one database relationship.
5. The owning side determines what relationship value JPA writes.
6. `mappedBy` points to a Java field on the owning entity, not a database column.
7. Both sides of a bidirectional relationship must be synchronized in Java.
8. Helper methods are the safest way to preserve relationship consistency.
9. Setting an entity reference does not automatically persist the referenced entity.
10. Cascading controls lifecycle propagation; it does not define relationship ownership.